

Stability and Correctness Testing of the Python Pickle Module

1 Introduction

The pickle module is a core tool in Python's standard library used for object serialization and deserialization. "Serialization" (pickling) refers to the process of converting a Python object with hierarchical structure into a byte stream, while "deserialization" (unpickling) refers to converting the byte stream back into the original object structure. Widely used in data persistence, cross-process communication, and model storage scenarios, pickle is an indispensable foundational module in the Python ecosystem.

Although pickle is powerful, in practice—especially in cross-environment and cross-end development—developers are often concerned with its stability and correctness. This report thus focuses on testing the following: given the same input object, will pickle always produce exactly the same output across different scenarios and environments?

pickle output consistency may be affected by factors such as the operating system, Python version, floating-point precision, and complex or recursive data structures. Therefore, we designed a set of pickle testing conditions that include both black-box and white-box testing strategies, covering type equivalence classification, boundary analysis, mock testing, and even source-level structural coverage testing (such as all-def, all-uses data flow analysis). With consistent inputs, we systematically analyze and test the stability and correctness of pickle.

This report will elaborate on the design rationale, testing strategy, traceability matrix, and key findings and limitations, and summarize the stability and correctness of pickle across multiple environments.

2 Black-Box Testing

The purpose of black-box testing is to assess and verify the stability and correctness of the pickle module from a user's perspective by comparing discrepancies in outputs under identical inputs. It also serves to identify potential implementation flaws or boundary condition issues to guide subsequent white-box testing.

To begin understanding the mechanism of pickle, we selected inputs that are completely identical and tested whether `pickle.dumps(obj)` always produces consistent serialized outputs (i.e.,

identical hash values), particularly under various data types and edge-case scenarios. To this end, we adopted the following four methods for testing: equivalence class partitioning, boundary value analysis, compatibility testing, and fuzz testing.

2.1 Equivalence Class Partitioning

By selecting representative object types, we simulate all common data scenarios that may be handled by pickle, and verify the serialization stability of pickle across different object types. This helps identify output inconsistencies caused by specific data types (such as custom classes or recursive structures). Meanwhile, the sampled data from this method can also serve as samples to test the differences in how pickle handles various data types across different environments and systems.

We constructed 16 equivalence classes covering:

- Basic atomic types: integer (42), float (3.14159), string ("hello world"), boolean (True), None
- Compound structures: list ([]), dictionary ({ "key": "value" }), tuple ((1, "a")), set ({1, 2, 3}), OrderedDict
- Special objects: recursive list (self-referential structure), custom class instances (including those with slots)
- Third-party library objects: datetime, Decimal, UUID
- International characters: strings with Unicode and emoji (e.g., "你好，世界🍀")
- Nested mixed structures: dictionary containing a list ({ "a": [1, { "b": 2 }] }), list containing a tuple ([(3, 4)])

2.2 Boundary Value Analysis

Since most issues occur at extreme boundaries, we designed test cases for data extremes, special states, and edge structures that pickle might encounter, directly verifying its stability under critical conditions.

Boundary cases include:

- Numeric extremes: system max integer (sys.maxsize), min integer (-sys.maxsize-1), Decimal('1E+1000')
- Empty objects: empty list ([]), empty dict ({}), empty string ("")
- Special float values: positive infinity (inf), negative infinity (-inf), NaN

- Deep structures: 50-level nested list (`([[[...]]])`)
- Large-scale data: list of 10,000 elements, 4KB byte stream
- Time extremes: earliest (`datetime.min`) and latest (`datetime.max`) supported by Python

2.3 Fuzz Testing

Fuzz testing uses automated methods to generate large amounts of random, complex input to test pickle's stability and robustness with unconventional and unpredictable data. Unlike boundary value analysis and equivalence partitioning, which use predetermined values, fuzz testing generates unstructured input data, simulating real-world unpredictable complex object structures. These complex, deeply nested, or mixed-type objects help uncover pickle's potential crashes, exceptions, or output inconsistencies in such cases, better reflecting real development scenarios where pickle is used to serialize data from complex sources.

Fuzz test objects feature:

- Deep nesting: 2–5 levels of randomly nested lists/dicts (e.g., `[{"data": [None, 591.47, [True]]}]`)
- Type mixing: forced random combinations of integers, strings, floats, None, booleans, NaN
- Special construction: sets and frozensets are converted to lists, NaN marked as strings
- Data scale: each object group contains 10 main elements, balancing efficiency and complexity

3 White-Box Testing

White-box testing focuses on coverage of the program's internal structure and implementation details, ensuring that pickle produces consistent serialization output across all implementation paths, branches, conditions, and data flows. This project adopts five classic white-box testing techniques:

3.1 Statement Coverage

Statement coverage ensures every executable statement is touched and verified during testing. For pickle, this helps discover unhandled data type scenarios and ensures all core processing branches can serialize and deserialize various inputs. High statement coverage maximizes control over pickle's functionality, directly supporting our "identical input, identical output" goal.

We mapped out all execution flows for pickle object handling, designing test inputs for different types, structures, and protocol branches to ensure each object type is fully serialized and

deserialized. We prioritized full coverage, paying special attention to boundary values, recursive structures, and special types to activate all implementation statements. Each coverage increases the depth of binary output consistency verification. These cases ensure all code lines are activated when handling complex structures, enabling repeated consistency checks across Python environments.

Example test methods:

- `test_recursive_list()`: recursive list structures
- `test_recursive_dict()`: recursive dict structures
- `test_custom_object()`: custom class objects

3.2 Branch Coverage

Branch coverage requires all decision points (e.g., if/else, protocol selection, type checks) to be tested, focusing on whether identical input still produces consistent pickle output under different branch implementations. For pickle, different branches, exception branches, or protocol branches may lead to different outputs. Comprehensive branch coverage helps verify "identical input yields identical hash pickle files under all branch implementations."

We mapped all major branch logic in pickle (e.g., protocol dispatch, custom serialization, exception handling), designing targeted test cases to repeatedly verify whether the binary output remains consistent for the same input object across environments and Python versions. Cases cover not only standard processing but also custom protocols and special types, aiming for comprehensive stability verification of pickle's implementation details.

Example test methods:

- `test_non_picklable_type()`: non-picklable types (exception branch)
- `test_custom_reduce()`: custom reduce protocol
- `test_dispatch_table()`: type dispatch branches

3.3 Condition Coverage

Condition coverage focuses on all possible values of each boolean condition expression, requiring each condition to be both True and False, thus more finely verifying whether minor condition changes affect pickle's binary output for the same input object.

We analyzed key compound conditions in pickle's source code, designing test inputs to cover all possible values for each condition (e.g., whether an object implements a protocol, existence of custom methods), and repeatedly verified output consistency for the same input under different

condition combinations. Only full coverage of all condition items can eliminate the risk of serialization results diverging due to minor implementation differences.

Example test methods:

- `test_custom_getstate_setstate()`: Condition evaluation for custom and non-custom getstate/setstate
- `test_custom_reduce()`: custom vs. non-custom reduce conditions

3.4 Path Coverage

Path coverage requires all possible execution paths (including complex nesting, recursion, and loops) to be tested. Pickle's internals handle recursive objects, nested structures, and circular references with different path combinations; path coverage is key to discovering "identical input but output affected by implementation path."

We designed numerous complex data structures (multi-level nesting, deep recursion, cross-references) to force pickle through all possible serialization and deserialization paths, serializing the same input multiple times across environments and strictly comparing the hash values of the resulting pickle files to find output differences caused by different implementation paths.

Example test methods:

- `test_recursive_dict()`: recursive self-referential dicts
- `test_cross_reference()`: cross-reference structures
- `test_deep_nested_list()`: deeply nested lists

3.5 Data Flow Coverage

Data flow coverage focuses on all definition (def) and usage (use) paths for each variable (such as object attributes). For pickle, different attribute assignments and accesses, dynamic attributes, and recursive references may affect serialization order and results, impacting binary consistency.

We constructed objects with dynamic attributes, repeated assignments, and recursive references, serializing and deserializing them repeatedly under different lifecycles, branches, and protocols, tracking each data flow path to verify the impact of attribute definition and usage on pickle output consistency. By forcing coverage of all assignment and access modes, we ensure pickle's output is consistent and stable under identical data flows.

Example test methods:

- `get_test_data()`: batch construction of multi-attribute objects, dynamic attributes,

recursive structures

4 Compatibility Testing

Pickle's serialization results may differ under different environmental factors. Compatibility testing compares outputs from identical test cases run on multiple Python versions and operating systems to further understand environmental impacts on pickle.

We tested the following environmental factors:

- Using the same pickle protocol (protocol 4), we serialized identical input objects under different mainstream Python versions (3.6, 3.7, 3.8, 3.10, 3.11), comparing the generated byte streams and hash values for consistency.
- Running identical Python versions and test cases on different operating systems (Windows, Linux), we observed whether serialization results differed, focusing on the effects of file paths, encoding, and underlying system calls.

5 Traceability Matrix

Test Objective/Requirement	Equivalence Partitioning Example	Boundary Value Example	Fuzz Testing Example	Compatibility Testing Example
Stability Verification: Identical input produces identical output (Python version/OS/other)	√	√	√	√
Correctness Verification: Objects are equal after serialization and deserialization	√	√	√	
Basic Type Processing: Integer, float, string, etc. primitive type serialization consistency	√			√
Compound Structure Processing: List, dict, tuple, etc. container type serialization consistency	√			√
Special Structure Processing: Recursive lists/dicts, custom class instance serialization stability	√			√
Extreme Value Processing: Very large/small integers, special float values		√	√	√
Empty Structure Processing: Empty list, empty dict, etc. serialization consistency	√	√		√
Deep Nesting Processing: Deeply nested list/dict serialization stability	√	√	√	√

International Character Processing: Unicode, emoji, etc. string serialization consistency	√			√
Large Data Volume Processing: 10,000-element list serialization performance and stability		√		√

6 Key Findings

Through black-box and white-box testing, we identified the following three issues:

6.1 Output Inconsistency Between Python 3.5/3.6 and Other Versions

Phenomenon: In Python 3.5 and 3.6, pickle's serialization output (hash value) for the same object differs from that of newer versions (3.7, 3.8, 3.11, etc.).

Reason: Early versions (Python 3.5/3.6) handle object references and memory layout differently, causing subtle differences in the serialized byte stream even with the same protocol, especially in attribute order, reference counting, and caching mechanisms.

6.2 Set Types Cause Output Differences Before and After Python 3.7

Phenomenon: Objects like `{"seg": {'c', 'a', 'b'}}` are serialized differently across Python versions.

Reason: Python's set type is unordered, with element order determined by hash values, insertion order, and memory allocation. Serialization cannot guarantee determinism. From Python 3.7 onward, the serialization logic for sets and other containers was optimized to automatically sort sets before serialization, resulting in outputs like `{"seg": {'a', 'b', 'c'}}`.

6.3 File Path Separators (“\” vs “/”) Cause Output Differences Between Windows and Linux

Phenomenon: Objects containing file paths (e.g., `"data/file.txt"`) produce different pickle outputs on Windows and Linux, mainly due to different path separators (“\” vs “/”).

Reason: Windows uses backslash (“\”) as a path separator, while Linux/macOS use slash (“/”). Pickle serializes the raw string as is, without converting separators, so the resulting byte streams and hash values differ.

7 Limitations and Recommendations

Although this testing systematically covered pickle's behavior across various environments and data types, the following limitations and areas for improvement remain:

Testing mainly covered mainstream Python versions (3.5, 3.6, 3.7, 3.8, 3.10, 3.11) and Windows/Linux systems. Some older or newer Python versions, macOS, and special hardware architectures (such as ARM) were not covered, so undiscovered compatibility issues may exist.

8 Conclusion

This project systematically evaluated the stability and correctness of the pickle module across different environments and diverse data types using equivalence partitioning, boundary value analysis, fuzz testing, and compatibility testing, employing both black-box and white-box techniques.

Test results show:

- Pickle's serialization output is highly dependent on Python version, operating system, protocol version, and internal object structure. Even with identical input, the output pickle file (hash value) may differ across environments.
- Main inconsistencies are found in early Python versions (e.g., 3.5/3.6), set types, and objects involving file paths across different operating systems.
- Pickle is designed to guarantee object equivalence, not cross-environment byte stream consistency. Therefore, pickle is unsuitable for scenarios requiring strict hash consistency or cross-platform, cross-version data exchange.

In summary, pickle is suitable for object persistence and inter-process communication within the same environment, but for scenarios requiring byte-level consistency or cross-environment data exchange, more stable and controllable serialization schemes (such as JSON, MessagePack, or custom protocols) are recommended.

Our Code: [1Ingeborg/Software-Testing-BTH014_2025-](https://github.com/Ingeborg/Software-Testing-BTH014_2025-)

Su Yuqin	Writing Code and Preparing the Report
Shen Chenyang	Writing Code and Preparing the Report
Xie Yuyang	Preparing the Report
Wang Conghong	Preparing the Report

```

return {
    "int": 42,
    "float": 3.14159,
    "str": "hello world",
    "unicode_str": "你好, 世界 🌍",
    "list": [1, 2, 3],
    "dict": {"key": "value", "num": 10},
    "tuple": (1, "a", 3.14),
    "seg": {'c', 'a', 'b'},
    "set": {1, 2, 3},
    "bool": True,
    "none": None,
    "nested": {"a": [1, {"b": 2}], "c": (3, 4)},
    "ordered_dict": OrderedDict([("a", 1), ("b", 2)]),
    "recursive_list": recursive_list,
    "custom_class": MyClass(),
    "datetime": datetime.datetime(2025, 5, 20, 10, 30),
    "decimal": decimal.Decimal("3.14159"),
    "uuid": uuid.UUID("12345678123456781234567812345678")
}

```

Figure 1: Detailed Data of Equivalence Classes

```
return {
    "max_int": max_int,
    "min_int": min_int,
    "zero": 0,
    "long_str": long_str,
    "empty_str": "",
    "empty_list": empty_list,
    "large_list": large_list,
    "empty_dict": empty_dict,
    "max_float": float('inf'),
    "min_float": float('-inf'),
    "nan_float": float('nan'),
    "decimal_max": decimal.Decimal('1E+1000'),
    "decimal_min": decimal.Decimal('-1E+1000'),
    "deep_nested_list": deep_nested_list,
    "long_tuple": tuple(range(1000)),
    "long_bytes": b'\x00' * 4096,
    "oldest_date": datetime.datetime.min,
    "future_date": datetime.datetime.max,
}
```

Figure 2: Detailed Data of Boundary Values

```

"fuzz_0": {
  "user": "k4i1a>",
  "active": true,
  "data": [
    null,
    [
      21
    ],
    "NaN",
    [
      82
    ],
    {
      "=[\n1R": null,
      "-a`}G": null,
      "mNxEA": "<J[\n#-uJdb",
      "@^paj": null
    }
  ]
},
"fuzz_1": {
  "user": "Oe&\fJr",
  "active": false,
  "data": [
    "e3kR?'Ffg-",
    "XB^F\nC~Gv&"
  ]
},
"fuzz_2": [
  "<@rHLd29_r",
  581
],

```

Figure 3: Detailed Data of Fuzzing

Name	Stmts	Miss	Cover
C:\Users\atrox\AppData\Local\Programs\Python\Python310\Lib\pickle.py	1160	872	25%
TOTAL	1160	872	25%

Figure 4: Code coverage of white-box test data in the pickle source code.

```
[INFO] Checking for cross-version hash consistency...
[X] MISMATCH with py310.txt: empty_tuple, fuzz_0, seg, fuzz_6, zero, empty_dict, fuzz_7, counter, empty_set, fuzz_1, dict_of_lists, int_zero, no
ne, fuzz_4, bool, fuzz_5, bool_false, large_dict, int, dict_keys_order1, bool_true, empty_list
[X] MISMATCH with py311.txt: empty_tuple, fuzz_0, seg, fuzz_6, zero, empty_dict, fuzz_7, counter, empty_set, fuzz_1, dict_of_lists, int_zero, no
ne, fuzz_4, bool, fuzz_5, bool_false, large_dict, int, dict_keys_order1, bool_true, empty_list
[X] MISMATCH with py36.txt: fuzz_0, seg, fuzz_6, fuzz_7, counter, fuzz_1, dict_of_lists, fuzz_4, fuzz_5, large_dict, dict_keys_order1
[X] MISMATCH with py37.txt: empty_tuple, fuzz_0, seg, fuzz_6, zero, empty_dict, fuzz_7, counter, empty_set, fuzz_1, dict_of_lists, int_zero, non
e, fuzz_4, bool, fuzz_5, bool_false, large_dict, int, dict_keys_order1, bool_true, empty_list
[X] MISMATCH with py38.txt: empty_tuple, fuzz_0, seg, fuzz_6, zero, empty_dict, fuzz_7, counter, empty_set, fuzz_1, dict_of_lists, int_zero, non
e, fuzz_4, bool, fuzz_5, bool_false, large_dict, int, dict_keys_order1, bool_true, empty_list
```

Figure 5: Specific data items showing hash mismatches between Python 3.5 and other versions (e.g., Python 3.6–3.11) during cross-version pickle serialization consistency checks.

```
[INFO] Checking for cross-version hash consistency...
[✓] MATCH with py311.txt
[✗] MISMATCH with py35.txt: int, seg, bool, none, zero, empty_list, empty_dict, fuzz_0, fuzz_1, fuzz_4, fuzz_5, fuzz_6, fuzz_7, int_zero, bool_
true, bool_false, empty_tuple, empty_set, dict_keys_order1, dict_of_lists, counter, large_dict
[✗] MISMATCH with py36.txt: int, seg, bool, none, zero, empty_list, empty_dict, int_zero, bool_true, bool_false, empty_tuple, empty_set
[✓] MATCH with py37.txt
[✓] MATCH with py38.txt
```

Figure 6: Cross-version pickle hash comparison — py310 matches py311, py37, py38; mismatches found with py35 and py36.

```
C:\Users\atrox\Desktop\软件测试
Original Object : 'C:\\Users\\atrox\\Desktop\\软件测试'
Pickled (hex) : 80049527000000000000008c23433a5c55736572735c6174726f785c4465736b746f705ce8bdaf4bbb6e6b58be8af95942e
SHA256 Hash : d22a8f653dd68458bdd5d329dced19a2b175b86c82f06a4fdbcf1c1db919e21f

(base) root@autodl-container-1a4d48a0d9-aa514cfe:~/autodl-tmp/软件测试# python path-test.py
/root/autodl-tmp/软件测试
Original Object : '/root/autodl-tmp/软件测试'
Pickled (hex) : 80049521000000000000008c1d2f726f6f742f6175746f646c2d746d702fe8bdaf4bbb6e6b58be8af95942e
SHA256 Hash : 6575d61f89033645493dfb6e85d8c864a6513fff15bfc39dbd97c812db743a0fc
```

Figure 7: Difference in system path separators between Windows (\) and Linux (/).

```
[✗] py35: Mismatched objects: fuzz_0, seg, tricky_platform, fuzz_6, fuzz_5, dict_of_lists, dict_keys_order1, counter, large_dict, fuzz_7, fuz
1, fuzz_4
[✗] py36: Mismatched objects: seg, tricky_platform
[✗] py37: Mismatched objects: seg, tricky_platform
[✗] py38: Mismatched objects: seg, tricky_platform
[✗] py310: Mismatched objects: seg, tricky_platform
[✗] py311: Mismatched objects: seg, tricky_platform
```

Figure 8: Inconsistent set ordering across Windows and Linux, even under the same Python version.